SCIENCEUTSAV®
Science is awesome

I'm a ROBO SCIENTIST

M

MOTOR AND ADVANCED CONTROL

Curiosity Kickoff

- What happens when we give power to a motor?
- Can a motor run faster or slower?
- Can a motor change direction?

Concept Connection

This topic introduces students to Looping Structures through the session Motor and Advanced Control. The goal is to keep the learning simple, practical, and beginner-friendly. Students first understand the concept in easy words, then see a small Arduino program, and then connect the code to the real output or input behavior.

1.Push Button

A tiny spring inside makes contact only when you press it. That's why your LED or buzzer works only while your finger is holding it down. Real-life: doorbells, calculators, game controllers.

2.Potentiometer

A knob connected to a resistive track. Turning it changes resistance, which changes voltage. This is how it dims a light or controls fan speed. Real-life: radio volume knobs, fan regulators, LED dimmers.

3.Limit Switch

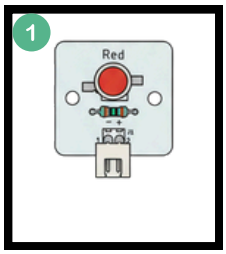
A mechanical switch with a small lever or button that clicks when pressed by an object. Often used for safety — like detecting if a door is closed before a machine runs. Real-life: microwaves, 3D printers, elevators.

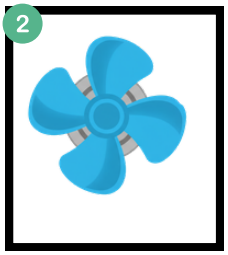
4.DC MOTOR

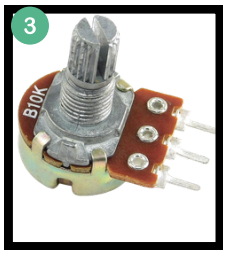
A DC Motor is an electrical device that converts direct current (DC) into mechanical motion. When powered, it rotates and is commonly used in fans, toys, and small machines.

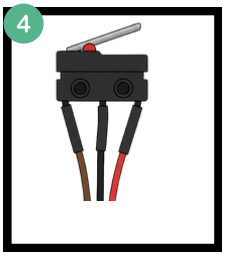
5.Watt SMD LED

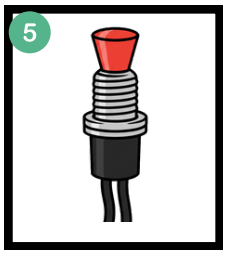
- I'm flat and powerful. Just one of me can light up your whole table like a torch.
- I don't like wasting energy, so I shine super bright without getting too hot.
- You've already met me I live in your mobile flashlight!











Beginner: DC Motor Basics

Class Explanation

In this class, students learn the basic idea of DC Motor Basics. The code below is kept simple so beginners can focus on understanding the main concept without extra complexity.

Materials Required

- Arduino with attached shield
- 2 × DC motor
- RMC wires
- USB cable

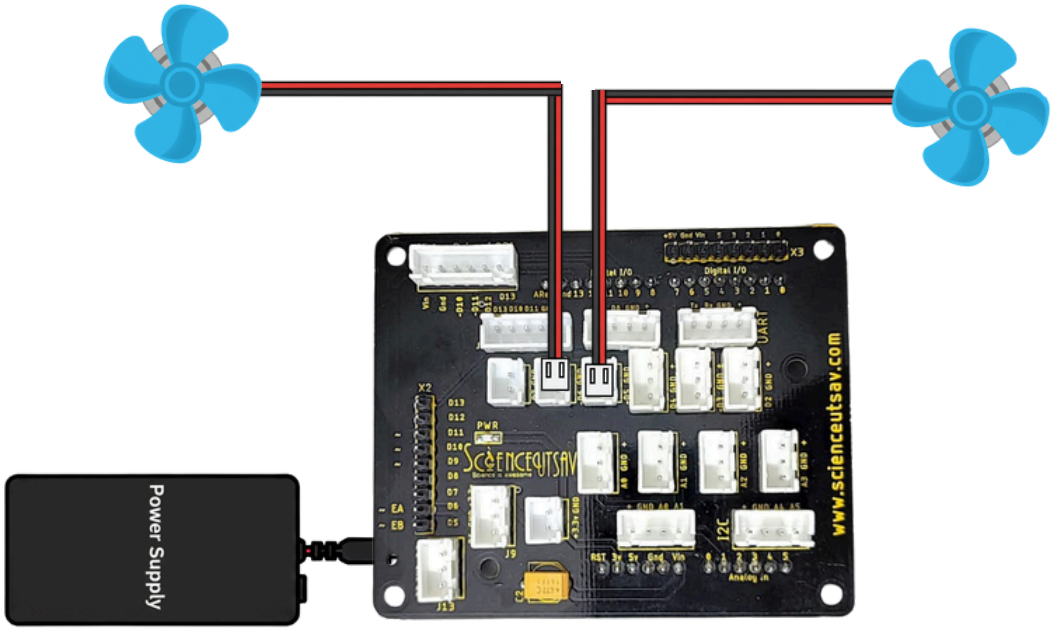
How to Connect

Use RMC wires to connect :

- 1 × DC motor → Arduino pin D5 and GND
- 1 × DC motor → Arduino pin D6 and GND

Then connect the Arduino to your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
const int motorPin1 = 5;
const int motorPin2 = 6;

void setup() \
 pinMode(motorPin1, OUTPUT);
 pinMode(motorPin2, OUTPUT);
\

void loop() \
 digitalWrite(motorPin1, HIGH);
 digitalWrite(motorPin2, LOW);
 delay(2000);
 digitalWrite(motorPin1, LOW);
 digitalWrite(motorPin2, LOW);
 delay(2000);
\
```

Code Explanation

- In an Arduino program, first we define the pins or variables to give names to pin numbers so they are easy to use in the code. For example, we can define an LED pin.

- Next, the setup() function runs only one time when the Arduino starts. It is used to prepare the pins and start things like the Serial Monitor. Inside setup, we use pinMode() to tell whether a pin will work as an input or an output.

- After that, the loop() function runs again and again continuously. This allows the Arduino to keep checking or controlling the circuit without stopping.

- Inside the loop, we use digitalWrite() to turn an output ON or OFF. For example, HIGH turns the LED ON and LOW turns it OFF.

So overall, setup() prepares everything once, and loop() keeps controlling the circuit continuously.

Try This Yourself Code

This Try This Yourself code keeps the same topic of DC Motor Basics but changes the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
const int motorPin = 5;

void setup() \
 pinMode(motorPin, OUTPUT);
\

void loop() \
 digitalWrite(motorPin, HIGH);
 delay(1000);
 digitalWrite(motorPin, LOW);
 delay(500);
\
```

Objective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learn

the basic idea of DC Motor Basics how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Intermediate: Motor ON OFF Control with Arduino

Class Explanation

In this class, students learn the basic idea of Motor ON OFF Control with Arduino. The code below is kept simple so beginners can focus on understanding the main concept without extra complexity.

Materials Required

- Arduino with attached shield
- 1 × Push button
- 1 × DC motor
- RMC wires
- USB cable

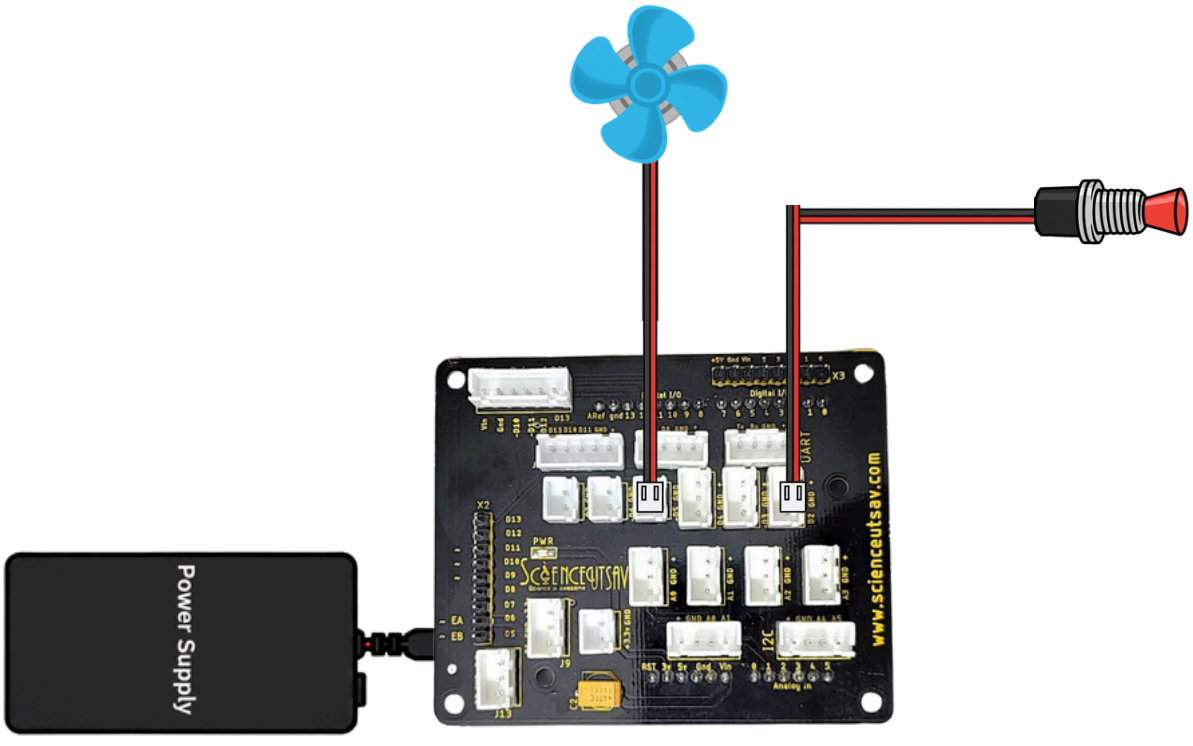
How to Connect

Use RMC wires to connect :

- 1 × Push button → Arduino pin D2 and GND
- 1 × DC motor → Arduino pin D5 and GND

Then connect the Arduino to your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
const int buttonPin = 2;
const int motorPin = 5;

void setup() \
 pinMode(buttonPin, INPUT_PULLUP);
 pinMode(motorPin, OUTPUT);
\

void loop() \
 if (digitalRead(buttonPin) == LOW) \
   digitalWrite(motorPin, HIGH);
 \ else \
   digitalWrite(motorPin, LOW);
 \
\
```

Code Explanation

- In an Arduino program, first we define the pins or variables to give names to pin numbers so they are easy to use in the code. For example, we can define a button pin and an LED pin.

- Next, the setup() function runs only one time when the Arduino starts. It is used to prepare the pins and start things like the Serial Monitor. Inside setup, we use pinMode() to tell whether a pin will work as an input or an output.

- After that, the loop() function runs again and again continuously. This allows the Arduino to keep checking inputs and controlling the circuit without stopping.

- Inside the loop, digitalRead() is used to check the state of a button or switch (HIGH or LOW). Based on this input, we use an if condition to make decisions. For example, if the button is pressed, we can turn an LED ON using digitalWrite(), otherwise we turn it OFF.

So overall, setup() prepares everything once, and loop() keeps reading inputs and controlling outputs continuously using simple conditions.

Try This Yourself Code

This Try This Yourself code keeps the same topic of Motor ON OFF Control with Arduino but changes the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
const int buttonPin = 2;
const int motorPin = 5;
const int ledPin = 6;

void setup() \
 pinMode(buttonPin, INPUT_PULLUP);
 pinMode(motorPin, OUTPUT);
 pinMode(ledPin, OUTPUT);
\

void loop() \
 if (digitalRead(buttonPin) == LOW) \
   digitalWrite(motorPin, HIGH);
   digitalWrite(ledPin, HIGH);
 \ else \
   digitalWrite(motorPin, LOW);
   digitalWrite(ledPin, LOW);
 \
\
```

Objective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learnctive:
the basic idea of Motor ON OFF Control with Arduino how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Advanced: Speed Control and Reverse Motor Logic

Class Explanation

In this class, students learn the basic idea of Speed Control and Reverse Motor Logic. The code below is kept simple so beginners can focus on understanding the main concept without extra complexity.

Materials Required

- Arduino with attached shield
- 1 × Potentiometer
- 2 × DC motor
- RMC wires
- USB cable

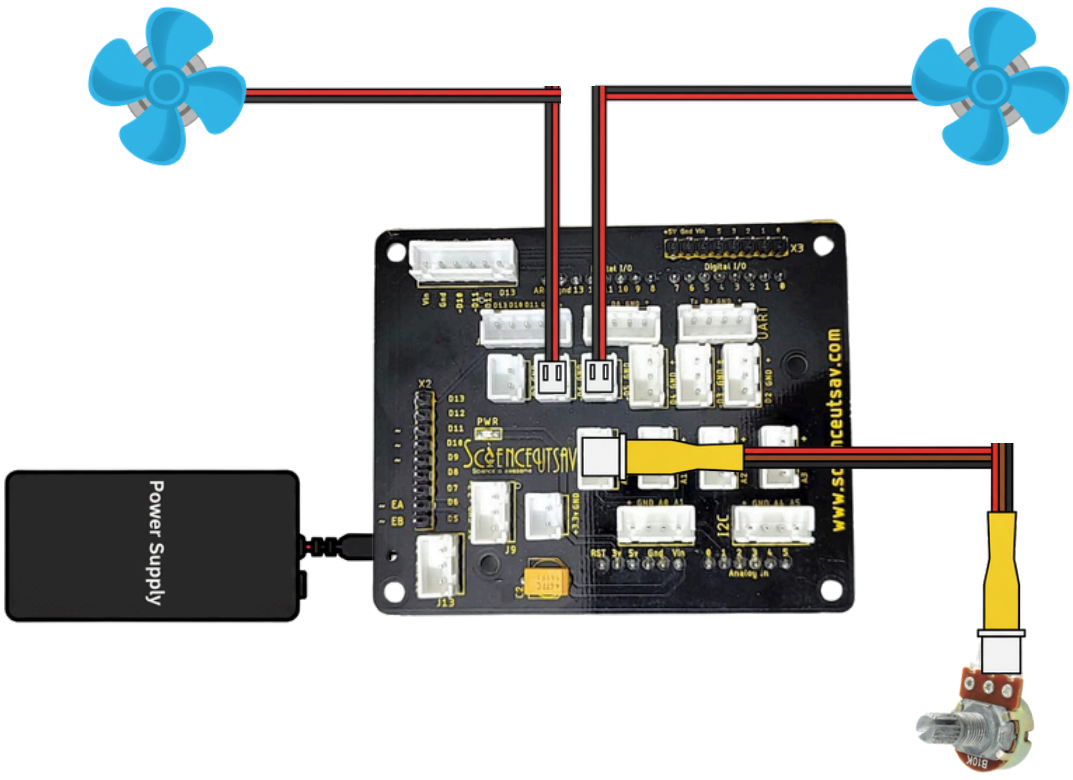
How to Connect

Use RMC wires to connect :

- 1 × Potentiometer → Arduino pin A0 and GND
- 1 × DC motor → Arduino pin D5 and GND
- 1 × DC motor → Arduino pin D6 and GND

Then connect the Arduino to your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
const int potPin = A0;
const int motorPin1 = 5;
const int motorPin2 = 6;

void setup() \
 pinMode(motorPin1, OUTPUT);
 pinMode(motorPin2, OUTPUT);
\

void loop() \
 int value = analogRead(potPin);

 if (value < 512) \
   analogWrite(motorPin1, map(value, 0, 511, 0, 255));
   digitalWrite(motorPin2, LOW);
 \ else \
   digitalWrite(motorPin1, LOW);
   analogWrite(motorPin2, map(value, 512, 1023, 0, 255));
 \
\
```

Code Explanation

- In an Arduino program, first we define the pins or variables to give names to pin numbers so they are easy to use in the code. For example, we can define a sensor pin and an LED pin.

- Next, the setup() function runs only one time when the Arduino starts. It is used to prepare the pins and start things like the Serial Monitor. Inside setup, we use pinMode() to tell whether a pin will work as an input or an output.

- After that, the loop() function runs again and again continuously. This allows the Arduino to keep checking inputs and controlling the circuit without stopping.

- Inside the loop, analogRead() is used to read changing values from sensors like a potentiometer or LDR. Based on these values, we use an if condition to make decisions. For example, if the value is high, we can turn an LED ON or control its brightness. We use digitalWrite() to turn the LED ON or OFF, and analogWrite() to control brightness or speed using PWM.

So overall, setup() prepares everything once, and loop() keeps reading inputs and controlling outputs continuously using simple conditions.

Try This Yourself Code

This Try This Yourself code keeps the same topic of Speed Control and Reverse Motor Logic but changes the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
const int potPin = A0;
const int motorPin = 5;

void setup() \
 pinMode(motorPin, OUTPUT);
\

void loop() \
 int value = analogRead(potPin);

 if (value < 341) \
   analogWrite(motorPin, 80);
 \ else if (value < 682) \
   analogWrite(motorPin, 160);
 \ else \
   analogWrite(motorPin, 255);
 \
\
```

Objective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learnctive:
the basic idea of Speed Control and Reverse Motor Logic how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Expert: Debugging Motor Control Interrupts Safety

Class Explanation

In this class, students learn the basic idea of Debugging Motor Control Interrupts Safety. The code below is kept simple so beginners can focus on understanding the main concept without extra complexity.

Materials Required

- Arduino with attached shield
- 1 × Limit Switch
- 1 × DC motor
- RMC wires
- USB cable

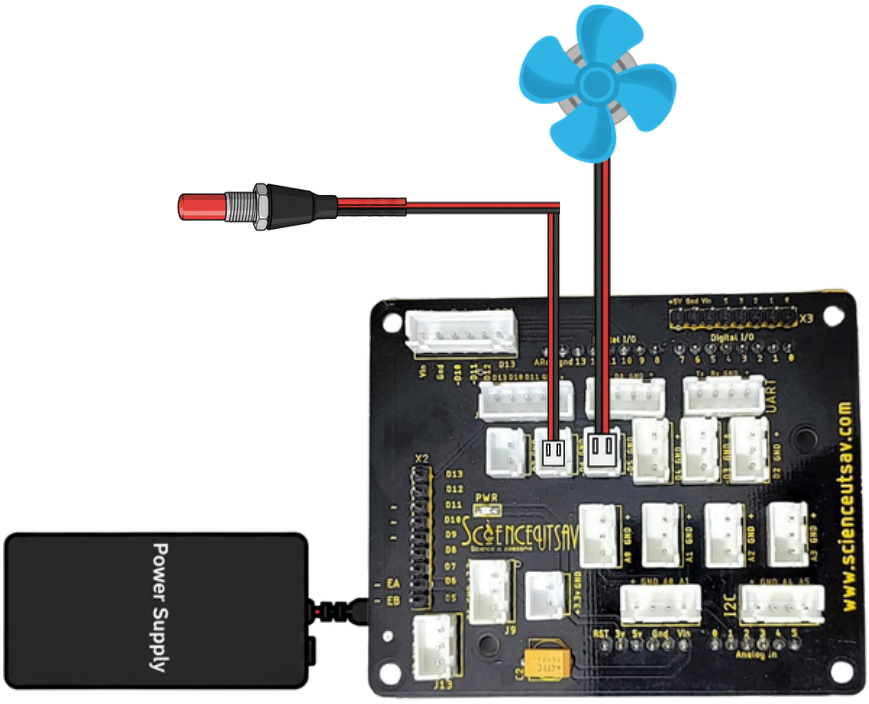
How to Connect

Use RMC wires to connect :

- 1 × Limit Switch → Arduino pin D2 and GND
- 1 × DC motor → Arduino pin D5 and GND

Then connect the Arduino to your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
#define BUTTON_PIN 6
#define MOTOR_PIN 5

bool motorState = false;
bool lastButtonState = HIGH;
bool currentButtonState;

void setup() {
  pinMode(BUTTON_PIN, INPUT_PULLUP); // Button connected between D2 and GND
  pinMode(MOTOR_PIN, OUTPUT);

  digitalWrite(MOTOR_PIN, LOW); // Motor OFF initially
}

void loop() {
  currentButtonState = digitalRead(BUTTON_PIN);

  // Detect button press (HIGH -> LOW)
  if (lastButtonState == HIGH && currentButtonState == LOW) {
    motorState = !motorState;   // Toggle motor state

    if (motorState) {
      digitalWrite(MOTOR_PIN, HIGH); // Motor ON
    } else {
      digitalWrite(MOTOR_PIN, LOW);  // Motor OFF
    }

    delay(200); // Simple debounce
  }

  lastButtonState = currentButtonState;
}
```

Code Explanation

- In an Arduino program, first we define the pins or variables to give names to pin numbers so they are easy to use in the code. For example, we can define a button pin and an LED pin.
- Next, the setup() function runs only one time when the Arduino starts. It is used to prepare the pins and start things like the Serial Monitor. Inside setup, we use pinMode() to tell whether a pin will work as an input or an output.
- After that, the loop() function runs again and again continuously. This allows the Arduino to keep checking inputs and controlling the circuit without stopping.
- Inside the loop, digitalRead() is used to check the state of a button or switch (HIGH or LOW). Based on this input, we use an if condition to make decisions. For example, if the button is pressed, we can turn an LED ON using digitalWrite(), otherwise we turn it OFF. We can also use Serial lines like Serial.println() to display values or messages in the Serial Monitor.

So overall, setup() prepares everything once, and loop() keeps reading inputs, controlling outputs, and showing information continuously.

Try This Yourself Code

This Try This Yourself code keeps the same topic of Debugging Motor Control Interrupts Safety but changes the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
const int limitSwitch = 2;
const int motorPin = 5;
const int ledPin = 6;

void setup() \
 pinMode(limitSwitch, INPUT_PULLUP);
 pinMode(motorPin, OUTPUT);
 pinMode(ledPin, OUTPUT);
 Serial.begin(9600);
\

void loop() \
 if (digitalRead(limitSwitch) == LOW) \
   digitalWrite(motorPin, LOW);
   digitalWrite(ledPin, HIGH);
   Serial.println("Safety stop active");
 \ else \
   digitalWrite(motorPin, HIGH);
   digitalWrite(ledPin, LOW);
   Serial.println("Motor allowed to run");
 \
 delay(500);
\
```

Objective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learnctive:
the basic idea of Debugging Motor Control Interrupts Safety how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Learning Outcomes
After completing this session, students should be able to:

- the basic idea of DC Motor Basics how the code connects with the circuit behavior how to upload and test a simple Arduino sketch
- the basic idea of Motor ON OFF Control with Arduino how the code connects with the circuit behavior how to upload and test a simple Arduino sketch
- the basic idea of Speed Control and Reverse Motor Logic how the code connects with the circuit behavior how to upload and test a simple Arduino sketch
- the basic idea of Debugging Motor Control Interrupts Safety how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Try Out Other Things

- use the Try This Yourself code after each main class code before asking students to build a completely new variation
- ask students to point out the exact lines that changed between the first code and the Try This Yourself codecode
- let students first run the Try This Yourself code code as it is and then change one value, one condition, or one output